HUST

ĐẠI HỌC BÁCH KHOA HÀ NỘI

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.

ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Classification of Malware Behavior Based on Known Malware Samples

ONE LOVE. ONE FUTURE.

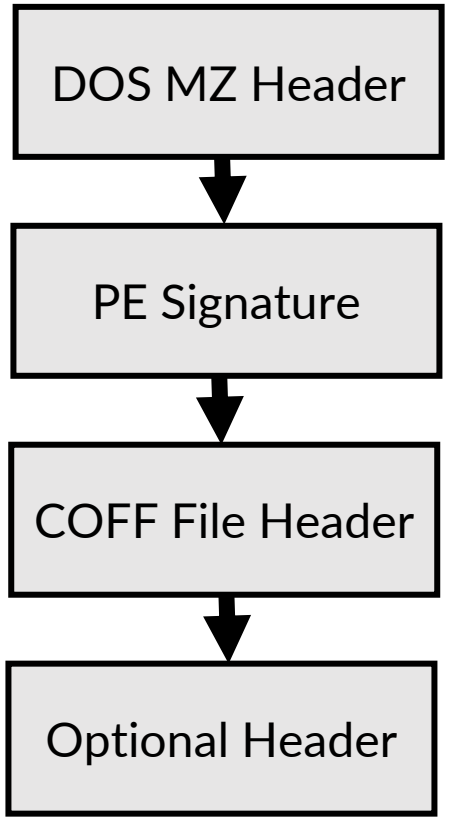
Members

- Đinh Phương Mai – 202417162
- Nguyễn Thành Công - 202416783

Table of Contents

1. Theoretical Background – PE Format & Disassembly

- **Headers:**



Responsible for backward compatibility

Indicates the beginning of NT file header

Specify machine types and number of sections

Contains entry point virtual address, image base, and directories

1. Theoretical Background – PE Format & Disassembly

- **Sections:**
  - .text or .code: Contains executable machine instructions
  - .data: Holds initialized global variables.
  - .rdata: Contains read-only variables, debug directories, and the Import Address Table (IAT).
  - .rsrc: Contains application resources (icons, menus, versions).

1. Theoretical Background – PE Format & Diassembly

- **Import Address Table (IAT):** Records the dynamic link libraries (DLLs) and functions imported from the system API (***LoadLibrary***, ***GetProcAddress***, ...)

- **Disassembly:** Reversing raw binary bytes in ***.text*** into x86 assembly mnemonics (***mov***, ***xor***, ***cmp***, ***etc***.) for opcode analysis.

1. Theoretical Background – Shannon Entropy

- For a discrete random variable *X* representing byte values (from 0x00 to 0xFF) in a binary file or section, the Shannon entropy *H*(*X*) is computed as:

$$H(X) = -\sum_{i=1}^{n} P(x_i) log P(x_i)$$

While n = 256 possible bytes values, $P(x_i)$ is the empirical probability of occurrence of byte x_i inside an analyzed block

- **Low entropy (0.0 – 5.0):** Highly structured, predictable data
- **Medium entropy (5.0 – 6.8):** Normal executable code or compiler output.
- **High entropy (7.0 – 8.0):** Indicates high randomness, approaching uniform distribution

→ This is typical of encrypted data, compressed buffers, or packing payloads, since cryptography and compression algorithms aim to eliminate statistical patterns.

1. Theoretical Background – Packing & Obfuscation

- **W^X Permission Violation:** Sections that are simultaneously Writable and Executable. Standard compilers separate these, but packers break this rule to decrypt code directly into memory.
- **Virtual vs. Raw Size Mismatch:** When the virtual size of a section in memory is significantly larger than its size on disk. This points to an expansion container for an unpacked payload.
- **Low/Zero Import Counts:** Packed files often hide their IAT, importing only baseline functions like ***LoadLibraryA*** and ***GetProcAddress*** to dynamically load APIs at runtime.
- **Suspicious Section Names:** Section headers named ***.upx0***, ***.aspack***, or containing unexpected characters like **/.**

1. Theoretical Background – Behavioral FSM: API Chain Tracking

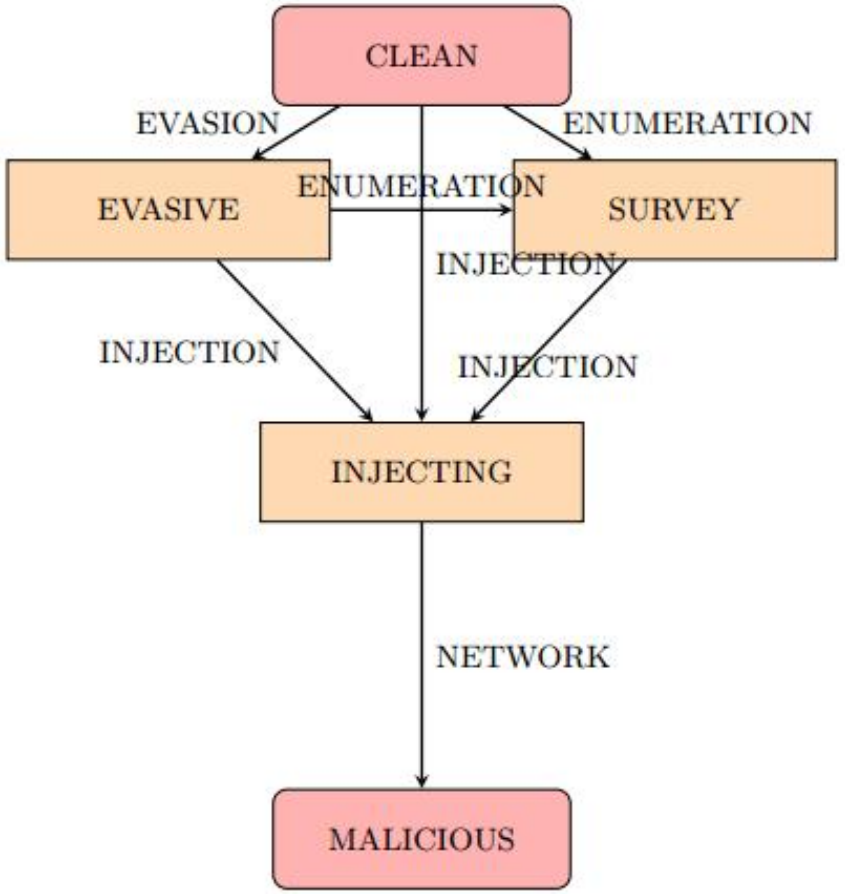


Figure 1: Behavioral FSM State Transitions

Score Impact:

CLEAN / EVASIVE: +0 pts

SURVEY: +5 pts

INJECTING: +15 pts

MALICIOUS: +35 pts

1. Theoretical Background – Decryption Loop FSM

- **State 1: START** → Scans disassembled assembly opcodes. If a decryption operation *(xor, rol, ror)* is detected, transition to **XOR_FOUND**.
- **State 2: XOR_FOUND** → If the subsequent instruction updates loop pointers *(cmp, test, dec, inc, add, sub)*, transition to **LOOP_FOUND**. Otherwise, reset.
- **State 3: LOOP_FOUND** → If the next instruction is a conditional jump or loop back *(jnz, jne, loop, jmp)*, transition to **DECRYPTOR**. Otherwise, reset.
- **Result:** Reaching the **DECRYPTOR** state signals a decryption stub, adding **+25** points to the heuristic score.

1. Theoretical Background – Weighted Heuristic Scoring Model

Table 2: Heuristic Scoring Weights and Indicators

Category	Anomalous Indicator	Score Weight
Code Section	Missing standard code section (`.text` renamed/packed)	+35
Entropy	Section entropy > 7.9 (encrypted/packed)	+45
	Section entropy > 7.5	+30
	Section entropy > 7.1	+15
Section Permissions	W^X violation (Writable + Executable section)	+35
Section Sizes	Virtual size vs Raw size mismatch (likely packed container)	+25
Section Naming	Known packed section name (e.g. `.upx0`, `.aspack`)	+35
	Section name starts with `/` (compiler anomaly/obfuscated)	+35
Overlay Details	Overlay size > 3 MB and file size > 4 MB	+55
	Overlay size > 150 KB and file size > 500 KB	+30
Opcode Density	Low opcode density (< 300 instructions in executable file)	+25
Imports Structure	Zero imports (extreme packers / shellcode)	+60
	Fewer than 5 imported functions	+30
Kernel Drivers	Kernel driver containing process control APIs	+50
Debug & Evasion	Advanced anti-debugging or debugging APIs	+30
Process Injection	Thread manipulation: Imports `SetThreadContext`	+20
	Dynamic API Resolution (`LoadLibrary` + `GetProcAddress` + `VirtualAlloc`)	+15
Instruction Ratio	Extremely high `add` opcode ratio (> 40%)	+40
	High `add` opcode ratio (> 30%)	+25
Strings	High-risk strings (commands targeting backups/AV exclusions)	+30

Dynamic Discounts (FP Mitigation):	
Export count > 50	-35
High DLL import count (>12)	-20
Windows API-Set DLL imports (> 4 ***api-ms-win**** libraries)	-20

➡ **Classification Threshold**: Files with a score ≥ 50 are classified as **MALWARE**.

1. Theoretical Background – Heuristic Probability Curve

To express threat likelihood as a percentage rather than a raw score, the engine computes a Malware Probability *P* using an exponential curve:

$$P = \begin{cases} 0.0 & \text{if } S \leq 0 \\ \left(\frac{S}{\text{Threshold}}\right) \times 50.0 & \text{if } 0 < S < \text{Threshold} \\ 50.0 + 50.0 \times \left(1 - e^{-0.05 \times (S - \text{Threshold})}\right) & \text{if } S \geq \text{Threshold} \end{cases}$$

While S is the heuristic score, Threshold = 50

➡ Scores that are far above the threshold asymptotically approach 100% malware probability.

2. System Architecture

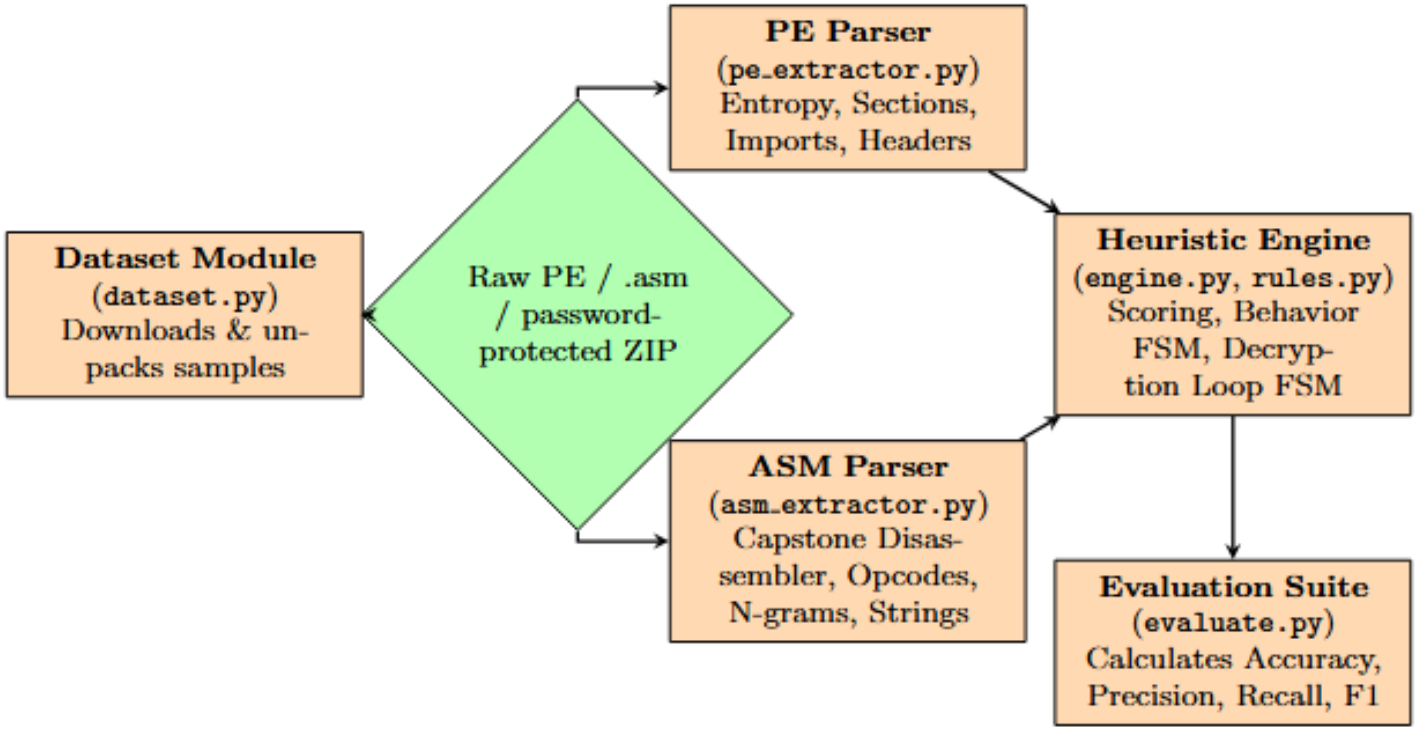


Figure 2: Malware Detection Architecture Flow

3. Evaluation

The performance of the heuristic engine was validated against a large-scale static features database containing 1,797 samples:

- **Benign Samples**: 993 legitimate Windows executables, DLLs, and system drivers.
- **Malware Samples**: 804 wild malware binaries collected securely from MalwareBazaar.

Metric	Optimized Heuristic Model
Accuracy	95.10%
Precision	94.53%
Recall (Sensitivity)	94.53%
F1-Score	94.53%

Table 4: Optimized Malware Detector Confusion Matrix ($N = 1,797$)

	Actual Malware	**Actual Benign**
Predicted Malware	760 (True Positive)	44 (False Positive)
Predicted Benign	44 (False Negative)	949 (True Negative)

4. Conclusion

- **Core strength**:
  - **Safety & Speed:** Evaluates files statically in milliseconds with zero threat execution risk.
  - **Robust Obfuscation Heuristics:** Catches packer stubs using entropy triggers and a custom 3-gram assembly FSM.

- **Drawbacks:**
  - **API Obfuscation Blindspot:** Unable to map API calls if resolved dynamically using custom hashes.
  - **Payload Encryption Limits:** Static analysis cannot parse hidden instructions before they are loaded into RAM.
  - **Dual-Use Tool Overlaps:** Legitimate debuggers and command engines naturally trigger injection rule

HUST

THANK YOU !